

Bases de datos y flujo de datos

Objetivos

Explicar cómo se construye una consulta, modelar source-transformación-validación-sink y distinguir datos de estructura.

Consulta y parámetros

Una base relacional recibe una sentencia que el motor tokeniza y analiza. Concatenar entrada dentro de esa sentencia permite que los caracteres del usuario cambien su estructura. Una consulta parametrizada envía estructura y valores por canales lógicos separados; el valor no se vuelve sintaxis SQL.

```
Inseguro: texto SQL + entrada -> parser SQL
Seguro:  sentencia con marcador -> parser/preparación
        valor separado          -> binding tipado
```

Parametrizar valores no resuelve por sí solo identificadores dinámicos como nombre de columna u orden. Para esos casos se selecciona entre valores permitidos definidos por la aplicación.

Modelo de flujo

- Source: punto controlable, por ejemplo parámetro HTTP, cookie, fichero o variable de entorno.
- Transformación: decodificación, concatenación, conversión, plantilla o canonicalización.
- Validación: condición que acepta o rechaza.
- Sink: operación sensible, como consulta, ejecución, apertura de archivo o petición de red.

Ejemplo:

```
query `id` -> URL decode -> conversión omitida -> concatenación SQL
           -> execute() -> filas/error/tiempo
```

Taint y confianza

Los datos no dejan de ser controlables por pasar por una variable interna. La pregunta es si una transformación garantiza la propiedad requerida por el sink. Escapar para HTML no garantiza seguridad para SQL; basenname no autoriza un objeto; Base64 no sanea una shell.

Prueba mínima

No empieces extrayendo datos. En un laboratorio, busca primero una diferencia reproducible que solo pueda explicar el parser relevante: error de sintaxis controlado, condición verdadera/falsa o tiempo, con controles negativos. Después identifica contexto, motor y número de columnas antes de ampliar impacto.

Reconstruir la consulta antes del payload

El endpoint `/items?id=7` devuelve un artículo. `id=7-0` devuelve el mismo; `id=7-1` devuelve el artículo 6; `id=7x` responde `400 integer required`.

Observación: algunas expresiones aritméticas parecen evaluarse, pero letras se rechazan. Qué sé realmente: el comportamiento es compatible con SQL numérico, pero también con un evaluador o conversión previa. Hipótesis: H1, la aplicación concatena `id` en SQL; H2, evalúa una expresión y luego parametriza el resultado. Experimento: comparar una pareja booleana válida para el motor sospechado y revisar el error de una construcción que el evaluador local aceptaría pero SQL no, manteniendo un control de ID inexistente. Resultado del escenario: `7 AND 1=1` devuelve el artículo y `7 AND 1=2` no devuelve filas; el log de laboratorio muestra `WHERE id = 7 AND 1=2`. Conclusión: existe un oráculo booleano en contexto numérico. Una comilla no era necesaria y habría probado un contexto distinto.

Contextos que no se parametrizan igual

Necesidad	Forma insegura	Tratamiento seguro
Valor de id	WHERE id = + entrada	marcador y binding entero
Texto de búsqueda	LIKE '% + entrada + %'	marcador y valor construido fuera de SQL
Columna de orden	ORDER BY + entrada	mapa cerrado name -> users.name

El tercer caso no se resuelve enviando el nombre de columna como valor parametrizado: se selecciona una expresión SQL ya definida por el programa.

Mitigación

- Consultas parametrizadas para valores.
- Tipos y allowlists para identificadores o decisiones dinámicas.
- Cuenta de base de datos con privilegio mínimo.
- Errores externos genéricos y logs internos suficientes.
- Pruebas unitarias y de integración sobre la frontera source-sink.

Referencias

- [OWASP SQL Injection Prevention Cheat Sheet](#)
- [Fuente interna SQLi](#)

Anterior: [Linux y shell](#). Siguiente: [Metodología](#).